

模块六: Web 应用架构

第二十三讲: 挂载硅基大脑——API 编排与请求闭环

[解释]

本讲概览

这是一节以"对接与联调"为核心目标的实操课。你将编写一段真实的 Python 脚本, 完成三次 API 调用闭环: 首先与竞技场大厅完成 Ping-Pong 握手, 接着调用硅基流动的大模型进行推理计算, 最后将 AI 生成的结果发送到竞技场大屏, 与其他同学一较高下。

它综合复习了 GET/POST、JSON 解析和 API 调用的知识。

一、本讲目标：串联两个外部服务

构建"取任务 → 模型处理 → 回传结果"完整数据链路

[解释]

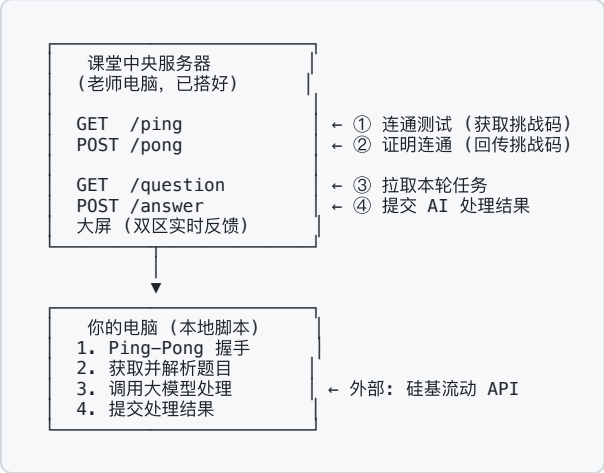
什么是 AI 竞技场

AI 竞技场是一个多维度的验证平台。在这里你的代码执行结果具有：(1) 社会可见性——你的输出会被全班看到，(2) 即时反馈——代码成功运行，名字立刻被点亮。

在这个过程中，你不仅是代码的编写者，更是接口的消费者。只有代码完全符合接口契约，才能成功登场。

系统概览：你的脚本要对接哪两个服务

整体架构



四次核心 API 调用

步骤	方法	接口	目标
① 握手	GET	/ping	获取服务器下发的随机挑战码
② 确认	POST	/pong	携带挑战码返回, 服务端验证通过后点亮大屏
③ 取题	GET	/question	获取正式的隐藏密令题目
④ 提交	POST	/answer	提交 AI 处理结果, 显示于大屏排行区

关键认知

你写的这段脚本就是一个最简 AI 智能体 (Agent) 流水线:

鉴权连通 → 接收任务 → 调用智能大脑 → 返回结果

这个结构和第 7 周要学的"AI 智能体"在拓扑上一致: 鉴权 → 接收任务 → 调用模型 → 返回结果。

[解释]

初窥智能体 (Agent) 架构

从架构层面理解, 你接下来要写的这几段脚本, 也是一个微型的"智能体"原型:

1. 本地 Python 脚本是执行主体, 负责控制整个流程。
 2. 课堂服务器是任务调度中心——下发任务、接收并评判结果。
 3. SiliconFlow 大模型是推理模块——负责对任务进行语义理解与处理。
- 了解这一点有助于为后续复杂的 Agent 开发建立直觉。

二、分步调通管道

三个 API，逐个击破

[解释]

分步调试原则

不要试图一口气写完所有代码。将完整的任务拆分为三个独立的步骤，确保每一步都能走通、能跑出预期结果后，再去写下一步。这是避免"代码写全—运行全报错"的最佳工程实践。

Step 1a: 接口规范——Ping-Pong 握手协议

接口文档

```
GET /ping
请求头: X-API-Key: <从智慧课堂获取的专属Key>
返回:   { "challenge": 42 }

POST /pong
请求头: X-API-Key: <专属Key>
请求体: { "response": 42 }
返回:   { "message": "验证通过, 大厅已点亮!" }
```

协议逻辑：服务器在 `/ping` 返回一个随机数（challenge），客户端必须将该值原样发回 `/pong`，以证明收发双向链路均正常。

身份认证铁律：永远不要把密钥直接写在代码里！通过环境变量（`os.environ`）加载是标准的安全规范。

参考提示词（发给 AI 助手）

"请帮我用 Python 的 requests 库实现一个 Ping-Pong 握手流程：

1. 先向 `{SERVER}/ping` 发送 GET 请求，请求头带 `X-API-Key`，拿到返回的 `challenge` 值。
2. 再向 `{SERVER}/pong` 发送 POST 请求，请求体为 `{"response": challenge值}`，同样带 `X-API-Key` 头。
3. API Key 必须从环境变量 `ARENA_KEY` 读取，不得硬编码在代码中。"

Step 1b：参考代码与跑通验证

AI 生成的参考代码

```
import requests
import os

SERVER = "https://syb.ncu.edu.cn/smart-class/api"
ARENA_KEY = os.environ["ARENA_KEY"] # 从智慧课堂网站获取

# 第一步：发送 GET /ping, 获取 challenge
ping_res = requests.get(
    f"{SERVER}/ping",
    headers={"X-API-Key": ARENA_KEY}
).json()
print(f"收到挑战码: {ping_res['challenge']}")

# 第二步：将 challenge 原样发回 POST /pong
pong_res = requests.post(
    f"{SERVER}/pong",
    headers={"X-API-Key": ARENA_KEY},
    json={"response": ping_res["challenge"]}
).json()
print(f"状态: {pong_res['message']}")
```

跑通后的预期输出

收到挑战码：42
状态：验证通过，大厅已点亮！

运行成功后，你的名字将出现在大屏幕左侧区域。

常见报错排障

报错	可能原因
KeyError: 'challenge'	终端未执行 <code>export ARENA_KEY="..."</code>
HTTP 401	Key 填错或未申请
Timeout / ConnectionError	未处于课堂同一网络

跑通本步骤 = 网络连通 ✓ + 环境变量配置 ✓ + JSON 收发正确 ✓。这是后续所有步骤的前提条件。

Step 2: 调用硅基流动 LLM API

回顾：硅基流动 API

大家在前面几周已经配置过 API Key。今天我们把它用在实战中。

代码

```
import requests
import os

SILICONFLOW_KEY = os.environ["SILICONFLOW_KEY"] # 强制要求通过环境变量配置密钥
SILICONFLOW_URL = "https://api.siliconflow.cn/v1/chat/completions"

def ask_llm(question_text, system_prompt="你是一个知识渊博的助手"):
    resp = requests.post(SILICONFLOW_URL,
        headers={"Authorization": f"Bearer {SILICONFLOW_KEY}"},
        json={
            "model": "Qwen/Qwen3.5-4B",
            "messages": [
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": question_text}
            ]
        })
    return resp.json()["choices"][0]["message"]["content"]

# 测试
answer = ask_llm("用一句话解释什么是 API")
print(f"AI 回答: {answer}")
```

代码解析

- **system_prompt**：控制 AI 的人格和回答风格。
 - **system_prompt** 决定了 AI 回答的约束范围与输出风格——这是 Prompt 工程的核心控制变量。
- **model**：使用 Qwen/Qwen3.5-4B，免费。
- **messages**：对话格式，和 ChatGPT 的结构完全一致。

验证

运行前，请先在终端配置你的系统环境变量：

```
export SILICONFLOW_KEY="sk-写上你的真实Key"
```

运行后终端应输出类似：

```
AI 回答：API 就是两个程序之间约定好的沟通方式，就像Web 服务的API 接口配置一样。
```

关键能力： `ask_llm` 函数把"调用大模型"封装成了一行代码。这就是工程中的"抽象"——把复杂操作简化为一个函数调用。

工程规范： API Key 永远不要明文硬编码在代码中，应从环境变量读取。

Step 3: 组装实战武器——打通问答链路

API 文档：正式业务接口

```
GET /question
请求头: X-API-Key: <从智慧课堂获取的专属Key>

返回:
{ "id": 1, "text": "真实题目隐藏在这里..." }

POST /answer
请求头: X-API-Key: <从智慧课堂获取的专属Key>

请求体:
{
  "question_id": 1,
  "answer": "AI 破译的最终结果",
  "system_prompt": "你使用的核心指令"
}
```

业务闭环逻辑

我们需要把你的 AI 引擎 (ask_llm) 与课堂的主线任务结合起来：

1. **拉取**：用 GET /question 从服务器获取本轮任务。
2. **处理**：调用 ask_llm()，通过 System Prompt 指定模型的角色与输出格式。
3. **提交**：用 POST /answer 将 AI 处理结果连同使用的 Prompt 一并发送给服务器。

💡 这三个步骤（拉取任务 → 模型处理 → 回传结果）与第 7 周“自主智能体 (Agent)”的核心交互流在结构上完全一致。

[解释]

控制大模型的输出

提交答案时必须展示你的 system_prompt，是因为大模型的实际表现是由 System Prompt 决定的。你能否让 AI 在满足角色设定的前提下，每次稳定输出符合业务规则的文本，考验的就是你设计系统重提示词的能力。

三、组装完整管道

三步合一，一键运行

[解释]

系统集成测试

将三个独立验证过的步骤合并为一个完整脚本，是软件工程中“集成测试”的简化版。通过这种“先分解、逐个验证、再合并”的方法论，能够让你在出错时迅速定位是自己的系统错误还是第三方接口故障。

完整管道脚本

arena_client.py

```
import requests
import os

# === 课堂服务器配置 ===
SERVER = "https://syb.ncu.edu.cn/smart-class/api"
ARENA_KEY = os.environ["ARENA_KEY"] # 从智慧课堂复制并 export

# === LLM 工具配置 ===
SILICONFLOW_KEY = os.environ["SILICONFLOW_KEY"]
SILICONFLOW_URL = "https://api.siliconflow.cn/v1/chat/completions"
MY_PROMPT = "你是一个知识渊博且幽默的助手"

# === LLM 工具函数 ===
def ask_llm(question_text):
    resp = requests.post(SILICONFLOW_URL,
        headers={"Authorization": f"Bearer {SILICONFLOW_KEY}"},
        json={
            "model": "Qwen/Qwen2.5-7B-Instruct",
            "messages": [
                {"role": "system", "content": MY_PROMPT},
                {"role": "user", "content": question_text}
            ]
        })
    return resp.json()["choices"][0]["message"]["content"]
```

arena_client.py (续)

```
# === 主流程 ===
# 步骤一：拉取题目
question = requests.get(f"{SERVER}/question", headers={"X-API-Key": ARENA_KEY}).json()
print(f"📄 题目: {question['text']}")

# 步骤二：调用大模型处理
print("正在调用大模型...")
answer = ask_llm(question["text"])
print(f"🖨️ 模型输出: {answer}")

# 步骤三：提交结果
result = requests.post(f"{SERVER}/answer",
    headers={"X-API-Key": ARENA_KEY},
    json={
        "question_id": question["id"],
        "answer": answer,
        "system_prompt": MY_PROMPT
    }).json()
print(f"✅ {result['message']}")
```

数据管道的完整闭环：该脚本串联了 GET 数据拉取、POST 数据汇聚、JSON 结构解析以及大模型多节点通信交互。

[解释]

结构与配置分离

完整的脚本分为清晰的段落：配置加载 -> 函数定义 -> 主流程执行。这种配置与业务逻辑相分离（将 API 密钥和 Server IP 等易变的常量放在文件顶部）的写法，是构建强健程序的标准工业规范。

运行管道脚本：确认跑通

操作

1. 确保 `ARENA_KEY` 与 `SILICONFLOW_KEY` 均已配置
2. 运行：`python arena_client.py`

预期终端输出（三步全通）

```
收到挑战码：42
状态：验证通过！
LLM 回答：API 是应用程序编程接口...
提交结果：{"status": "ok", "score": 1}
```

报错排障

现象	原因
卡在第一步	网络不通，或 <code>ARENA_KEY</code> 未 export
LLM 无响应	<code>SILICONFLOW_KEY</code> 未 export 或 余额不足
<code>score: 0</code>	提交格式不符，调整 System Prompt 后重试

终端出现"提交结果"且 score 非零 = 三步管道（身份验证 → LLM 调用 → 回传服务端）全部打通。

四、实战演练：结构化输出约束

结构化输出——约束大模型的响应格式

实操任务：从自然语言中提炼意图

题目获取方式

题目不在幻灯片上，必须通过接口拉取：

```
question = requests.get(
    f"{SERVER}/question",
    headers={"X-API-Key": ARENA_KEY}
).json()
```

拉取后你会看到类似：

"我明天要去趟北京，麻烦帮我瞅瞅那边的天气，千万别下暴雨啊"

任务要求

- 从这段口语化文本中，精准提取核心实体与意图
- 服务端只接受极简格式，例如：北京-天气
- 符合格式 → 服务端判定成功；
多余字符 → 判定失败

为何这样设计？ 真实的 Agent 间通信不允许废话。后端服务只消费结构化参数，多余的自然语言会直接导致下游解析失败。

System Prompt 设计：输出格式控制

问题：大模型默认喜欢"说废话"

用宽泛的指令，模型依然会加前缀和解释：

"好的，我来帮您提取一下，
客户想去北京..."

这一个字也不会被服务端接受。

✗ 无效 Prompt

"你是一个数据提取专家，请帮我分析这段话。"

身份太宽泛，模型会正常说话。

✓ 有效 Prompt

"你是一个自然语言意图解析引擎。
请直接提取【目标城市】和【查询动作】，
用减号连接。
禁止输出除结果以外的任何字符。"

三个约束要素：

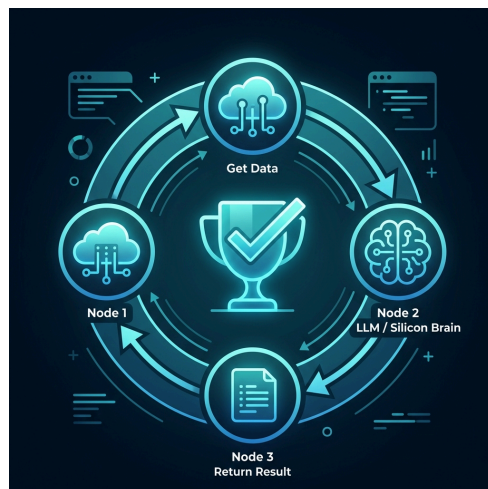
1. 角色定义：引擎，不是助理
2. 格式规则：城市-动作
3. 负向约束：禁止多余输出

结构化输出控制是智能体开发的核心技能：Agent 间的通信必须是纯净的结构化数据，不允许自然语言填充。

课程小结

今日关键收获

1. **API 消费全流程**：GET 取数据 + POST 提交数据（对接课堂服务器）。
2. **LLM API 调用**：用 Python 调用硅基流动大模型，封装为可复用的 `ask_llm` 函数。
3. **完整管道**：获取任务 → 调用 AI 工具 → 返回结果——这就是智能体的雏形架构。
4. **Prompt 工程实战**：通过 System Prompt 控制 AI 的输出风格。



下节课预告

管道已经通了。下节课我们将进入 HOTL 实战——指挥 AI 完成结构化预测任务。你将面对真实的结构化业务数据。你需要：

- ⚡ 快速阅读接口文档 → 设计提示词指令 → 调用大模型 → 将结果传回大屏
- 🧠 根据返回的报错信息，不断优化你的 System Prompt
- 🏆 争取在大屏幕的实时排行榜上取得最高分！

这也是第 8 周黑客松实战的综合预演——提前适应多步 API 联调与提示词迭代优化的完整流程！

[解释]

开发能力进阶

从本节课开始，你的代码不再仅仅是自娱自乐的本地计算。你构建的应用能够感知外部环境、调用极其强悍的外部计算中心（硅基大脑），并与其他服务节点进行分布式通信。